



HOSPITALINE SYSTEM MANUAL

TERESA RODRÍGUEZ
GUILLERMO LOZANO
DIANA BRAVO

We have created a Database for Hospitals call *HOSPITALINE*.

The program has two main packages that have code.

Db.pojos

First we have db.pojos wich contains the pojos Patient, Doctor, Appointment, MedicineSupply, MedicalRecords, User and Role.

User

- *Purpose*: represents a user entity stored in the *users* table (table in the Database).
- *Serializable*: implements Serializable which means that can be saved/transferred as a byte stream, use to be able to work with JPA and JAXB.
- Has an auto-generated primary key (id).
- Includes typical user fields (the attributes): username, password, and email.
 - email is marked as unique (@Column(unique =true)).
- Linked to a Role via a @ManyToOne relationship (roleId as the foreign key).
- Uses eager fetching for the role (loaded immediately with the user).

Role

- *Purpose*: represents a role entity stored in the *roles* table (table in the Database).
- *Serializable*: implements Serializable which means that can be saved/transferred as a byte stream, use to be able to work with JPA and JAXB.
- Has an auto-generated primary key (id) and a name field (that can be Doctor or Patient).
- Contains a list of users via a @OneToMany relationship.
 - This is lazy loaded (users are loaded only when accessed).
- Method addUser(User user) adds users to the role.

Patient

- *Purpose*: Represents the patient in our system
- *Serializable*: implements Serializable which means that can be saved/transferred as a byte stream, use to be able to work with JPA.

Attributes:

- id: Unique identifier
- name, email, address: Personal info
- sex: Gender
- dob: Date of birth (Date)

Constructors:

- Empty constructor.
- Constructors with and without id.

Methods:

- Standard getters/setters, to be able to work with JPA.
- toString() for readable output
- equals() and hashCode() based on id, the primary key attribute to be able to work with JPA.

Doctor

- *Purpose*: Represents a doctor in our healthcare system.
- *Serializable*: implements Serializable which means that can be saved/transferred as a byte stream, to be able to work with JPA.

Attributes:

- id: Unique identifier.
- name, email, address: Personal and contact details.
- dob: Date of birth.
- experience: Years of professional experience.
- department: Medical specialty (e.g., Cardiology, Pediatrics)
- sex: Gender.

Constructors:

- Empty constructor.
- Full constructor with id.
- Constructor without id (likely for creating new records before ID is generated).

Methods:

- Standard getters and setters for all attributes, to be able to work with JPA.
- toString(): Returns a readable string of all attributes.
- equals() and hashCode() based on id, the primary key attribute to be able to work with JPA.

Appointment

- *Purpose*: Represents a scheduled appointment between a patient and a doctor.
- *Serializable*: Can be stored/transferred as a byte stream, to be able to work with JPA and JAXB.
- Annotated with @XmlElement – supports XML marshalling/unmarshalling (e.g., for REST APIs with JAXB).

Attributes:

- id: Unique identifier for the appointment
- patientId: ID of the patient
- doctorId: ID of the doctor
- date: Date of the appointment
- hour: Hour of the appointment (likely in 24h format)

Constructors:

- Full constructor with id
- Constructor without id (for creating new appointments)
- Default constructor (empty constructor to be able to work with JAXB).

Methods:

- Standard getters and setters, to be able to work with JPA and JAXB.
 - Annotated with @XmlElement for XML support.
 - getDate() uses @XmlJavaTypeAdapter(SQLDateAdapter.class) to handle SQL date formatting in XML.
- equals() and hashCode(), based on id to be able to work with JPA.
- toString(): Returns a readable string of all attributes.

MedicalRecord

- *Purpose*: Represents a patient's medical record for a specific visit/appointment or treatment.
- *Serializable*: Can be serialized for storage or transmission, to be able to work with JPA and JAXB.
- Annotated with @XmlRootElement: Supports XML marshalling/unmarshalling (e.g., for web services or APIs).

Attributes:

- id: Unique identifier for the medical record.
- patientID: ID of the patient (foreign key).
- diagnosis: The medical diagnosis.
- treatment: Treatment or procedure applied.
- date: Date of the visit or diagnosis.
- medicineID: ID of the medicine prescribed (foreign key).

Constructors:

- Constructor with id.
- Constructor without id (for new entries).
- Default constructor (empty constructor to be able to work with JAXB).

Methods:

- Standard getters and setters, to be able to work with JAXB and JPA.

- Getters annotated with @XmlElement for XML output.
- getDate() uses @XmlJavaTypeAdapter(SQLDateAdapter.class) for correct XML date formatting.
- equals() and hashCode(), based only on id to be able to work with JPA.
- toString():

MedicineSupply

- *Purpose*: Represents a medicine item and its stock quantity in our medical inventory system.
- *Serializable*: Can be transferred/stored as a byte stream, to be able to work with JPA and JAXB..
- Annotated with @XmlRootElement: Ready for XML serialization (e.g., in RESTful or SOAP web services).

Attributes:

- id: Unique identifier for the medicine supply.
- name: Name of the medicine (e.g., Paracetamol, Amoxicillin)
- quantity: defines the quantity of the medicine (e.g., 100g, 4000g)

Constructors:

- Default constructor (empty constructor to be able to work with JAXB).
- Constructor with name and quantity (for new records).
- Constructor with id, name, and quantity (for existing records).

Methods:

- Getters with @XmlElement annotation for XML output and setters for each field, to be able to work with JAXB and JPA.
- equals() and hashCode(), based only on id, to be able to work with JPA.
- toString(): outputs a readable summary of the medicine and its quantity.

hospital

The program second main package is hospital, inside it we can find another five packages: ifaces, jdbc, jpa, ui and xml.

ifaces:

Inside ifaces we have three interfaces that serve the purpose of organizing the code into contracts for modularity, abstraction, and reusability.

HospitalManager

This interface provides a comprehensive set of methods for hospital system operations, including booking and managing appointments, handling doctor and patient info, managing medical records, and dealing with medicines.

Key Functionalities:

1. Appointment Management

- EliminateAppointment(int id, int patid): Allows a patient to cancel an appointment by ID.
- ModifyAppointment(int apoid, Appointment apo): Modify details of an existing appointment.
- BookAppointment(Appointment apo): Book a new appointment.
- CheckAvailability(int hour, int docid, Date date): Check if a doctor is available at a certain hour on a specific date.
- AddAppointmentsfromXml(ArrayList<Appointment> apos, int patid): Add multiple appointments from XML data for a specific patient.
- getAppointmentsPatient(int id): Get all appointments for a specific patient.
- getAppointmentsDoc(int docid): Get all appointments for a specific doctor.

2. Doctor Management

- AddDoctor(Doctor doc): Add a new doctor.
- ViewDoctorInfo(int docidsee): Get info about a specific doctor.

- ViewAllDoctors(): Get a list of all doctors.
- getDoctorByEmail(String email): Find a doctor by email.
- getPatientsByDoctor(int docid): List patients assigned to a particular doctor.
- DeleteDoctor(int id): Remove a doctor from the system.

3. Patient Management

- AddPatient(Patient pati): Add a new patient.
- getPatientByEmail(String email): Find a patient by email.
- DeletePatient(int id): Remove a patient.
- validatePatientOfTheDoctor(int docid, int patid): Check if a patient is under a certain doctor's care.

4. Medical Records Management

- ViewAllMedicalRecords(int patid): Get all medical records for a patient.
- ViewOneMedicalRecord(int medrecid, int patid): View a specific medical record.
- getMedicalRecordsByPatient(int patid): Get all medical records by patient.
- addMedicalRecord(MedicalRecord record): Add a new medical record.
- VerifyMedicalRecord(int medrecid, int patid): Verify ownership or validity of a medical record.
- updateMedicalRecord(MedicalRecord medrec): Update existing medical records.

5. Medicine Management

- ClaimMedicine(int medrecidclaim, int patid): Claim medicine related to a medical record.
- getAllMedicines(): Get a list of all medicines.
- addMedicine(MedicineSupply med): Add a new medicine supply.

- AddMedicinesfromXml(ArrayList<MedicineSupply> meds): Add multiple medicines from XML.
- isMedicineAvailable(int medId): Check if a medicine is available.

UserManager:

This interface is focused on managing users and their roles, mainly handling authentication, role management and user account maintenance. This interface defines the essential user management operations for our system, including user login, role-based access control, user registration, and account management tasks like password changes and deletion. It handles security-related functions like authentication and authorization by managing roles.

Key Functionalities:

1. Authentication:

- login(String name, String password): Authenticates a user by checking if the username and password exist and match in the database. Based on the user's role, it will determine which menu to show (doctor's or patient's).

2. Role Management:

- getRoles(): Retrieves a list of all roles defined in the system (e.g., patient, doctor).
- createRole(Role role): Creates a new role in the system.
- getRole(String name): Fetches a specific role by its name.

3. User Account Management:

- register(User user): Registers a new user in the system.
- assignRole(User user, Role role): Assigns a role to a user, which defines their permissions or access level.
- deleteAccount(User user): Deletes a user account from the system.
- changePassword(User user, String newPassword): Changes the password for a given user.

XmlManager

This XmlManager interface handles converting between Java objects and XML (and HTML) representations specifically

for appointments and medicine supplies. It essentially provides methods for marshalling and unmarshalling between these formats. This interface focuses on data interchange between Java objects and external file formats (XML and HTML) for appointments and medicines. We use it for importing/exporting data.

What each method does:

1. Appointments XML Marshalling and unmarshalling

- `Java2XmlAppointments(ArrayList<Appointment> apos):`
Converts a list of Java Appointment objects into XML format (marshal Java objects to XML).
- `Xml2JavaAppointments()` throws `UnmarshalException`: Reads appointments from an XML file and converts them back into Java Appointment objects (unmarshal XML to Java).

2. Medicines XML Marshalling and unmarshalling

- `Java2XmlMedicines(ArrayList<MedicineSupply> meds):`
Converts a list of Java MedicineSupply objects into XML.
- `Xml2JavaMedicines()` throws `UnmarshalException`: Reads medicines from an XML file and converts them back into Java objects.

3. Medicines HTML Export

- `Java2HTMLMedicines(ArrayList<MedicineSupply> meds):`
Converts a list of medicines into an HTML format, for display purposes.

jdbc:

First, JDBC is a Java API (Application Programming Interface) that allows Java programs to connect to and interact with databases. It provides methods to:

- Connect to a database.
- Execute SQL queries and updates.
- Retrieve results from the database.
- Handle database transactions.

HospitalManagerImplements

Is the only class inside this package, here we have the code of all the methods define in HospitalManager.

jpa:

JPA is a Java specification for managing, persisting, and accessing data between Java objects/classes and a relational database. It provides a standardized way to map Java objects to database tables and vice versa, making it easier to work with databases in Java applications without writing lots of SQL code.

UserManagerImplements

Is the only class inside this package, here we have the code of all the methods define in UserManager.

ui:

Stands for User Interface. Inside this package we have the Menu (main).

menu

This class allows users to register as doctors or patients, log in, and interact with appointments, medical records, and account management.

Main Functionalities:

1. User Registration:

- registerDoctor() and registerPatient():
 - Collect user info (email, name, password, etc.) from console.
 - Validate inputs (non-empty, valid dates, correct sex input).
 - Create user and specific entity (Doctor or Patient).
 - Register user and assign the appropriate role.
 - Add doctor/patient data to the hospital system.
 - Handle errors (e.g., duplicate emails, date format issues).

2. Login:

- Validate credentials.

- Redirect users to their respective menus based on role:
 - Doctors get the doctorMenu().
 - Patients get the patientMenu().

3. Patient Menu:

- Manage appointments: view, book, cancel, modify.
- View doctors: list all or view specific doctor details.
- Manage medical records: view all or single, claim medicines.
- Export/import appointments in XML.
- Change password.
- Delete account.
- Exit to main menu.

4. Doctor Menu:

- Manage appointments: view all appointments or patients with appointments.
- Manage medical records: view, add, modify for patients.
- Manage medicine supply: export/import in XML or HTML.
- Change password.
- Delete account.
- Exit to main menu.

Error Handling:

- Input validations with user-friendly messages.
- Try-catch blocks for IO, SQL, parsing, and other exceptions.
- Specific checks for unique email constraint.

xml:

AppointmentsList

Is a simple wrapper class designed for XML marshalling and unmarshalling of a list of Appointment objects.

What it does:

- Holds a list of Appointment objects in a private field appointments (an ArrayList).
- The class is annotated with `@XmlElement(name="Appointments")`, meaning when converted to XML, it will be the root element `<Appointments>`.
- The getter method `getAppointment()` is annotated with `@XmlElement(name="Appointment")`, so each individual Appointment object in the list will be represented as an `<Appointment>` element inside the `<Appointments>` root element in the XML.
- The setter `setAppointment()` allows setting the list when deserializing from XML.

Purpose:

This class is used with JAXB (Java Architecture for XML Binding) to convert between Java objects and XML documents. It allows us to marshal (write) or unmarshal (read) a collection of Appointment objects to/from XML format easily.

MedicinesList

Works exactly like the AppointmentsList class but for a list of MedicineSupply objects instead.

What it does:

- Holds a list of MedicineSupply objects in a private `ArrayList<MedicineSupply>` called medicines.
- Annotated with `@XmlElement(name="Medicines")` so when converted to XML, the root element will be `<Medicines>`.
- The getter method is annotated with `@XmlElement(name="Medicine")`, which means each MedicineSupply in the list will be represented as a `<Medicine>` element inside `<Medicines>` in the XML.
- The setter allows setting the list during XML deserialization.

Purpose:

This class is a container to marshal/unmarshal (convert) a list of MedicineSupply objects to and from XML using JAXB.

SQLDateAdapter

Is a custom JAXB adapter to convert between java.sql.Date and its XML string representation in the format "yyyy-MM-dd".

What it does:

- Extends XmlAdapter<String, Date>, which means it converts from Date (Java) to String (XML) and vice versa.
- Uses DateTimeFormatter with the pattern "yyyy-MM-dd" to format and parse dates.

Methods:

- marshal(Date date)
Converts a java.sql.Date to a formatted String for XML output.
 - Calls date.toLocalDate() to get the LocalDate part of the SQL Date.
 - Formats the date using the formatter into "yyyy-MM-dd" string.
- unmarshal(String datestr)
Converts a date string from XML back into a java.sql.Date.
 - Parses the string into a LocalDate using the same formatter.
 - Converts LocalDate back to java.sql.Date via Date.valueOf(LocalDate).

Purpose:

This adapter is used to customize how a java.sql.Date field is represented in XML during JAXB marshalling/unmarshalling.

XmlManagerImplements

Is an implementation of the XmlManager interface that handles converting between Java objects and XML files, specifically for Appointment and MedicineSupply objects. It also includes functionality to transform XML into HTML using XSLT.

What each method does:

1. Java2XmlAppointments(ArrayList<Appointment> apos)

- Converts a list of Appointment objects to an XML file named Appointments.xml.
- Uses JAXB marshalling to serialize the list wrapped in an AppointmentsList container.

2. Java2XmlMedicines(ArrayList<MedicineSupply> meds)

- Converts a list of MedicineSupply objects to an XML file named MedicinesExp.xml.
- Uses JAXB marshalling with a MedicinesList wrapper.

3. Xml2JavaMedicines()

- Reads and unmarshals the XML file MedicinesImp.xml back into a Java ArrayList<MedicineSupply>.
- Uses JAXB unmarshalling.
- Throws UnmarshalException if unmarshalling fails.

4. Xml2JavaAppointments()

- Reads and unmarshals the XML file Appointments.xml back into a Java ArrayList<Appointment>.
- Uses JAXB unmarshalling.
- Throws UnmarshalException if unmarshalling fails.

5. Java2HTMLMedicines(ArrayList<MedicineSupply> meds)

- Converts the list of MedicineSupply objects to XML (stored in MedicinesExpHTML.xml).
- Applies an XSLT transformation (MedicinesStyle.xslt) to convert the XML into an HTML file (Medicines.html).
- Uses Java's Transformer classes for XSLT processing.